

Kaiming 初始化

深度神经网络每一层大致是：

x 输入，经过线性变换 $z = Wx + b$ ，再经过激活函数 $a = \text{ReLU}(z)$

如果初始化不好，随着层数加深，会出现两个问题：

前向传播时，激活值越来越小。

[第 1 层输出正常；第 5 层输出接近 0；第 20 层输出几乎全是 0]

这会导致后面的层学不到有效特征。

反向传播时，梯度越来越小或越来越大。

[梯度太小：梯度消失，前面的层几乎不更新；梯度太大：梯度爆炸，训练不稳定]

所以初始化的目标是让每一层输出的方差尽量保持稳定，也就是希望：

$\text{Var}(\text{第 } l \text{ 层输入}) \approx \text{Var}(\text{第 } l \text{ 层输出})$

这样信号在网络中传播时不会逐层放大，也不会逐层衰减。

假设某一层有 n 个输入 $x_1, x_2, \dots, x_n$

输出某个神经元 $z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$

也就是 $z = \sum w_i x_i$

我们做几个常见假设

$E[w_i] = 0$

$E[x_i] = 0$

w_i 和 x_i 相互独立

不同输入之间近似独立

那么 z 的方差是 $\text{Var}(z) = \text{Var}\left(\sum w_i x_i\right)$

如果每一项近似独立 $\text{Var}(z) = \sum \text{Var}(w_i x_i)$

因为 w_i 和 x_i 独立，并且均值近似为 0，所以 $\text{Var}(w_i x_i) = \text{Var}(w_i) \text{Var}(x_i)$

假设每个权重的方差一样 $\text{Var}(w_i) = \text{Var}(W)$ ；

每个输入的方差一样 $\text{Var}(x_i) = \text{Var}(x)$

于是 $\text{Var}(z) = n \text{Var}(W) \text{Var}(x)$

这里的 n 就是输入维度，也叫：**fan_in**

所以线性层会把输入方差乘上 $n \text{Var}(W)$

如果我们希望线性层之后方差不变，就希望 $n \text{Var}(W) = 1$ ，也就是 $\text{Var}(W) = \frac{1}{n}$ 。

这其实接近 Xavier 初始化的思想。但是 ReLU 会改变这个结论。

ReLU 定义为 $\text{ReLU}(z) = \max(0, z)$

[假设 z 的分布是关于 0 对称的，比如近似正态分布。那么大约一半的值会被 ReLU 置为 0。

例如：

$$z = [-2, -1, 0.5, 1.5]; \text{ReLU}(z) = [0, 0, 0.5, 1.5]$$

所以 ReLU 会让信号能量大约损失一半。]

$$\text{从二阶矩角度看: } E[\text{ReLU}(z)^2] \approx \frac{1}{2} E[z^2]$$

也就是说，经过 ReLU 后，信号强度大约变成原来的一半。

刚才我们知道线性层会让方差变成 $\text{Var}(z) = n \text{Var}(W) \text{Var}(x)$

$$\text{然后经过 ReLU, 大约损失一半 } \text{Var}(a) \approx \frac{1}{2} \text{Var}(z)$$

$$\text{代入 } \text{Var}(a) \approx \frac{1}{2} \cdot n \cdot \text{Var}(W) \cdot \text{Var}(x)$$

$$\text{我们希望 } \text{Var}(a) \approx \text{Var}(x), \text{ 所以 } \frac{1}{2} \cdot n \cdot \text{Var}(W) \cdot \text{Var}(x) = \text{Var}(x)$$

$$\text{两边消掉 } \text{Var}(x): \frac{1}{2} \cdot n \cdot \text{Var}(W) = 1, \text{ 得到 } n \cdot \text{Var}(W) = 2$$

$$\text{所以 } \text{Var}(W) = \frac{2}{n}, \text{ 这就是 Kaiming 初始化公式: } \text{Var}(W) = \frac{2}{\text{fan_in}}$$

$$\text{如果用标准差表示 } \text{Std}(W) = \sqrt{\frac{2}{\text{fan_in}}}$$

$$\text{所以对于 ReLU 网络, 权重通常初始化为 } W \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in}}\right)$$

$$\text{或者写成标准差形式 } W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fan_in}}}\right)$$

$$\text{更严谨地说, 正态分布写法应该是 } W \sim \mathcal{N}(0, \text{std}^2), \text{ 所以 } \text{std} = \sqrt{\frac{2}{\text{fan_in}}}$$

fan_in 指的是某一层每个神经元接收多少个输入。对于全连接层 Linear:

```
nn.Linear(in_features, out_features)
```

fan_in = in_features; fan_out = out_features

[例如:

```
nn.Linear(512, 256)
```

那么 fan_in = 512; fan_out = 256]

$$\text{Kaiming 初始化中 } \text{Var}(W) = \frac{2}{512}, \text{ Std}(W) = \sqrt{\frac{2}{512}}$$

卷积层的权重形状通常是[out_channels, in_channels, kernel_height, kernel_width]

[例如:

```
nn.Conv2d(3, 64, kernel_size=3)
```

它的权重形状是[64, 3, 3, 3]

对于每一个输出位置，一个卷积核会看 3 个输入通道；每个通道 3 × 3 个像素。

所以 $\text{fan_in} = \text{in_channels} \times \text{kernel_height} \times \text{kernel_width}$
也就是 $\text{fan_in} = 3 \times 3 \times 3 = 27$

所以 Kaiming 初始化中 $\text{Var}(W) = \frac{2}{27}$, $\text{Std}(W) = \sqrt{\frac{2}{27}}$

Kaiming 正态初始化就是 $W \sim \mathcal{N}\left(0, \frac{2}{\text{fan_in}}\right)$

除了正态分布，也可以用均匀分布。

如果 $W \sim \mathcal{U}(-\text{bound}, \text{bound})$

均匀分布的方差是 $\text{Var}(W) = \frac{\text{bound}^2}{3}$

Kaiming 初始化要求 $\text{Var}(W) = \frac{2}{\text{fan_in}}$, 所以 $\frac{\text{bound}^2}{3} = \frac{2}{\text{fan_in}}$

得到 $\text{bound} = \sqrt{\frac{6}{\text{fan_in}}}$, 所以 Kaiming 均匀初始化为:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{\text{fan_in}}}, \sqrt{\frac{6}{\text{fan_in}}}\right)$$

Leaky ReLU 定义为如果 $x > 0$, $f(x) = x$; 如果 $x \leq 0$, $f(x) = ax$ 。其中 a 是负半轴斜率, 例如 $a = 0.01$; ReLU 是 Leaky ReLU 的特殊情况 $a = 0$, 对于 Leaky ReLU, 负半轴并不是完全变成 0, 而是乘上 a 。

这时增益 gain 变成 $\text{gain} = \sqrt{\frac{2}{1+a^2}}$

所以权重标准差是 $\text{std} = \frac{\text{gain}}{\sqrt{\text{fan_in}}}$, 也就是 $\text{std} = \sqrt{\frac{2}{(1+a^2)\text{fan_in}}}$

Xavier 初始化主要适用于

1. tanh
2. sigmoid
3. 线性激活

它的目标是让前向和反向都尽量稳定。

Xavier 的典型方差是: $\text{Var}(W) = \frac{2}{\text{fan_in} + \text{fan_out}}$

或者某些情况下 $\text{Var}(W) = \frac{1}{\text{fan_in}}$

但 Xavier 没有专门补偿 ReLU 会丢掉一半负值的问题。ReLU 会让大约一半神经元输出为 0, 所以需要把权重方差放大 2 倍。

因此凯明初始化是 $\text{Var}(W) = \frac{2}{\text{fan_in}}$

BatchNorm 会对中间激活进行归一化，让激活值均值接近 0，方差接近 1。

用了 BatchNorm，还需要凯明初始化吗？

仍然需要。原因是 BatchNorm 可以缓解激活分布漂移，但初始化仍然影响训练初期的稳定性。

如果初始化太差，网络一开始就可能出现：激活极端；梯度异常；训练非常慢；loss 不稳定。

所以常见 CNN / ResNet 依然会搭配：Conv + Kaiming 初始化 + BatchNorm + ReLU